

ParkNCharge with Nav2

This Project uses:

- Gazebo Ignition Fortress
- ROS2 Humble
- Nav2

The main part of the Project is built from 5 folder.

Launch folder:

[simulation.launch.py](#)

This ROS 2 launch script initializes a TurtleBot3 simulation in the Ignition Gazebo environment, setting up necessary environment variables, spawning the robot, and including multiple other launch files for simulation time, state publishing, and navigation. It dynamically loads resources like models and worlds, using the robot model specified in the TURTLEBOT3_MODEL environment variable.

[navigation2.launch.py](#)

The navigation2.launch.py includes the main Nav2 bring up launch file and configures Rviz with the default navigation view. Additionally, this launch file initializes the Bot with a specified map and a parameter file.

[robot_state_publisher.py](#)

The robot state publisher launch file creates a node which then can publish the robot's state in the simulated environment.

[ros_ign_bridge.py](#)

The ros_ign_bridge.launch.py bridge translates messages between ROS2 and Ignition Gazebo and sets up a static transform between the map and odom frames, enabling localization and perception. The static transform is there to define a fixed spatial relationship between both coordinate frames, which is needed for consistency.

Maps Folder:

The maps folder contains two different worlds. One empty world and one turtlebot3_world file. Each .yaml file is connected to a corresponding .pgm file. More about these files later on.

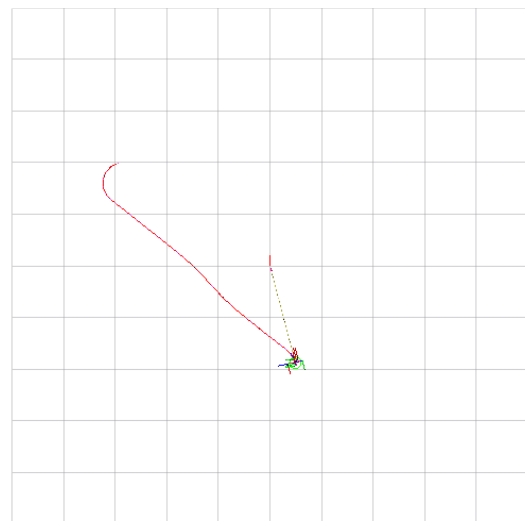
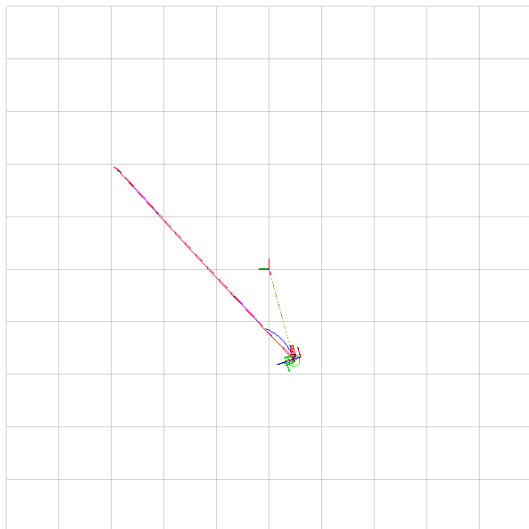
Models Folder:

The model's folder contains two subfolders. One containing the model and each of its parts and textures. And the other one containing a plain world. And a file which can add obstacles. The obstacles in 'model.sdf' in our case are congruent to the '.pgm' world image.

Params Folder:

The params folder contains two parameter files for Nav2. One which configures the robot to create a Path only valid for Ackerman steering with the corresponding constraints. The other one creates a direct Path to the goal with no restrictions.

you can see the difference between the two paths if the robot should have the same alignment to the car on the side. On the one with no restriction, it just drives to the goal in the shortest path and then turns on the spot whereas on the one with Ackermann constraints it must plan the final alignment from the start. In both images the car should look towards the east. If you play around with the param file you can even increase and decrease the turning radius at the end depending on the needs and wants. Here there is the same example but with obstacles.



Nav2

Advantages of Nav2

The architecture of Nav2 is highly modular and designed to operate with both simulated and real-time data. It supports a wide array of use cases, including simple goal-directed navigation, waypoint following, and advanced recovery behaviors. Designed with extensibility in mind, it enables developers to plug in their own custom algorithms or behaviors without needing to modify the core stack. This modularity is one of the reasons Nav2 has become a standard in the ROS 2 ecosystem. Its separation of concerns also allows teams to work in parallel on different aspects—planning, control, localization—without interference.

Now let's delve into the components that make up the Nav2 stack. The core modules include the BT Navigator, AMCL for localization, the Planner Server, Controller Server, and Velocity Smoother—each handling a key aspect of decision-making and motion. These are supported by utility modules like the Map Server and Saver for map management, the Behavior Server for handling fallback actions when navigation encounters issues, and the Waypoint Follower, which enables navigation through a sequence of predefined goals. Together, they form a system capable of autonomous navigation.

BT-Navigator

Starting with the Behavior Tree Navigator, it serves as the central decision-making unit. Nav2 employs behavior trees to organize and control complex sequences of actions. In our case, we use a predefined behavior tree structure defined in an XML file named `navigate_to_pose_w_replanning_and_recovery.xml`. This tree encapsulates high-level logic such as goal setting, path planning, path following, and recovery behaviors. One of the strengths of using behavior trees is that they are readable, modular, and allow for easy debugging and extension. The BT Navigator selects appropriate planners and controllers based on the scenario and includes logic to retry navigation attempts if failures occur. The tree has built-in failovers, enabling the robot to perform recovery actions like spinning in place, updating the goal, or switching to a different controller.

The behavior tree includes both fallback and sequential nodes. Fallback nodes allow the robot to try multiple strategies for a task, while sequence nodes enforce strict order. This allows for flexible and robust task execution. Additionally, the behavior tree runs in a loop, checking every few milliseconds whether goals have been reached or if recovery actions should be triggered. This frequency is configurable using the `bt_loop_duration` parameter.

Adaptive Monte Carlo Localization (AMCL)

AMCL, or Adaptive Monte Carlo Localization, is responsible for determining the robot's position on a known 2D map. It uses a particle filter approach, where multiple hypotheses (particles) are maintained for the robot's position. Each particle is updated using motion data from odometry and observations from the laser scanner. The system compares the predicted sensor readings of each particle with the actual sensor data to assign a likelihood score. Over time, the particles converge towards the robot's true position.

The AMCL module interfaces with multiple ROS topics such as `/scan`, `/map`, `/odom`, and `/tf`. It publishes the estimated pose on `/amcl_pose` and provides a cloud of particles representing various pose hypotheses. This continuous update loop ensures that even small shifts in position or orientation are quickly detected and corrected.

The process begins by receiving a map of the environment, which is used as a reference throughout the algorithm. The algorithm operates in two main update phases: the sensor update and the motion update.

In the sensor update, the algorithm simulates laser scans from the perspective of each particle using the known map. These simulated scans are then compared to the real sensor scan received from the robot's laser rangefinder. The comparison determines how closely each particle's predicted observation matches the actual observation, and based on this similarity, the algorithm assigns a score (or weight) to each particle — higher scores for better matches.

Parallel to this, the motion update begins by extracting the robot's movement from odometry data. This motion estimate is not perfectly accurate, so random noise is added to reflect uncertainty in movement. Each particle is then moved according to the noisy motion estimate, thereby updating its pose.

Once both updates are done, the system has a cloud of possible poses, represented by the weighted set of particles. These are then refined by resampling, focusing on particles with higher weights (more likely poses) and discarding unlikely ones. This refined cloud represents the most probable area of the robot's current location. Finally, AMCL publishes the estimated pose and the full particle cloud, allowing the robot to know where it is and enabling other modules (like planners or controllers) to work based on this estimate.

Parameters like the number of particles, beam skipping, and sensor noise models can be finely tuned to improve performance. For example, in our setup, we configured AMCL with a particle range of 500 to 2000 and used a "likelihood_field" model for the laser scans. 'Likelihood_field' means, the nearer the laser endpoints are the higher the likelihood that the pose is correct. The `alpha1` to `alpha5` values define the noise in the laser scans to simulate realistic errors in motion. Beam skipping thresholds and beam error distances were also adjusted for efficiency.

Planner Server

The Planner Server is another crucial component that determines how the robot finds its way from the current location to the target. In our implementation, we used the SmacPlannerHybrid plugin. This planner is kinematics-aware, meaning it considers the robot's motion constraints. Unlike traditional grid-based A* planners, the SmacPlannerHybrid takes orientation into account, planning in a three-dimensional space of x, y, and theta. This makes it particularly suitable for non-holonomic robots like those with differential or Ackermann steering. The planner first sets up the costmap, defines start and goal nodes, checks for collisions along the way, and applies a path smoother to refine the trajectory. The output is a ROS-compatible path message that is then passed to the controller.

A* Explanation Video: <https://www.youtube.com/watch?v=-L-WgKMFuhE&t=173s>

Controller Server

Once a path is planned, it's handed over to the Controller Server. The controller is responsible for converting the path into velocity commands that can be sent to the robot's motors. We used the DWBLocalPlanner plugin, which implements the Dynamic Window Approach.

The DWBLocalPlanner also uses additional plugins like the SimpleProgressChecker and SimpleGoalChecker. This helps determine whether the robot is making progress toward the goal and whether it has reached its destination. If progress stalls, the system can trigger recovery behaviors to get back on track.

The algorithm creates all possible paths which are possible in a specific time frame. Each Path is a Prediction of a Tuple of velocity and angle. The possible paths are constrained to the distance it can reach in a fix set time or obstacles in the way.

It then assigns scores based on several criteria. Each critic contributes a weighted cost to a given trajectory. The path with the lowest cost is selected, meaning the one which balances all goals. Critics we used where:

GoalDist: this encourages trajectories that get closer to the goal position

RotateToGoal: when the robot is close to the goal and needs to face towards it, this critic becomes essential.

BaseObstacle: It scores trajectories based on how close they get to known obstacles.

PathDist: focuses on the distance from the global path

Oscillation: In penalizes zig zagging or unstable paths

GoalAlgin: This critic ensures that the path ends in a direction that aligns well with the final goal heading ... we decided to prefer this more than others for the Ackermann constrained path.

PathAlgin: favors trajectories that follow the global paths heading

Velocity Smoother

The Velocity Smoother sits between the controller and the robot's motor interface. Its job is to smooth out sudden changes in velocity that might result from abrupt decisions by the controller. This not only improves the comfort of motion but also protects hardware from wear and tear. The smoother operates in closed-loop mode, meaning it considers actual feedback from the robot's movement. It adapts based on previous velocity commands and ensures that transitions in speed and direction are gradual and controlled.

The smoother's configuration can include parameters like `scale_velocities`, which determines whether to cap velocity at configured limits. If `scale_velocities` is set to false, the velocity smoother will not scale the speed commands based on the robot's maximum speed. Instead, it will smooth the raw velocity values directly. The smoother treats all velocities the same, regardless of how close they are to the max speed.

How does the Robot move?

The controller server just returns a `cmd_vel` message. We are using the libignition gazebo diff drive system plugin. This needs information like wheel separation, wheel radius and from where it must subscribe for movement commands.

Wheel Separation: $L = 0.287m$

Wheel Radius: $r = 0.033m$

To understand the robot's movement, we start with an example command velocity, which specifies a linear velocity of 0.3 m/s and an angular velocity of 0.5 rad/s. We calculate the linear velocities of the left and right wheels using the differential drive equations:

$$v_r = v + w * \frac{L}{2} = 0.3 + 0.5 * \frac{0.287}{2} = 0.37175 \frac{m}{s}$$
$$v_l = v - w * \frac{L}{2} = 0.3 - 0.5 * \frac{0.287}{2} = 0.22825 \frac{m}{s}$$

These linear speeds are then converted to angular speeds for the wheels:

$$w_r = \frac{0.37175}{0.033} \approx 11.266 \frac{rad}{s}$$

$$w_l = \frac{0.22825}{0.033} \approx 6.916 \frac{rad}{s}$$

Finally, these calculated angular velocities are passed to the robot's simulation plugin. This plugin applies the actual physics to move the robot in the simulated environment.

Real World Scenario

To move from simulation to reality, we need some basic hardware:

- A motor driver, which controls the power going to the motors.
- And a microcontroller with PWM-capable GPIO pins — because we'll be using Pulse Width Modulation to control motor speeds.

Now, it's important to remember real motors don't take linear velocity directly. They're limited by RPM — or revolutions per minute. For example, let's assume our motors have a maximum RPM of 100.

From earlier calculations, we already know the angular velocities:

- For the left wheel: about 6.916 rad/s, which converts to approximately 66 RPM.
- For the right wheel: about 11.266 rad/s, which becomes around 108 RPM — exceeding the 100 RPM limit!

To handle this in practice, we use PWM signals.

Here, 0 RPM corresponds to a PWM value of 0, and 100 RPM maps to the maximum value of 255.

$$RPM_r = 11.266 \frac{rad}{s} * \frac{60}{2\pi} \approx 108 \text{ RPM}$$

$$RPM_l = 6.916 \frac{rad}{s} * \frac{60}{2\pi} \approx 66 \text{ RPM}$$

That means for our left wheel, 66 out of 100 RPM means:

$$PWM_l = \frac{66}{100} * 255 \approx 168$$

For the right wheel, we cap the RPM at 100, so even though it wants to go at 108 RPM, we get:

$$RPM_r = \min\left(\frac{108}{100}, 1\right) * 255 = 255$$

PWM

PWM stands for Pulse Width Modulation, and it's a way of controlling how much power we deliver to a device — in our case, the motors. By rapidly switching the motor on and off and adjusting the ratio of “on” time to “off” time — called the duty cycle — we control the effective speed. A PWM value of 255 means full speed, and 0 means the motor is off.

Behavior Server

If the robot encounters an issue during navigation—say, it gets stuck or can't turn towards its goal—the Behavior Server takes over. This module is responsible for short-term, reactive behaviors. It contains plugins for actions like spinning, backing up, driving on heading, and waiting. These behaviors are triggered when normal navigation is no longer possible. For example, if no valid path is available, the robot might drive on heading. These fallback behaviors are crucial for ensuring that the robot can recover from temporary problems without human intervention.

Each behavior is implemented as a plugin and can be finely tuned. For instance, we defined maximum and minimum rotational velocities, acceleration limits, and simulated time horizons. These parameters ensure that recovery behaviors are safe and effective. The behavior server can also be extended to include custom behaviors such as assisted teleoperation, where the robot waits for remote control input when it encounters an unresolvable issue.

Waypoint Follower

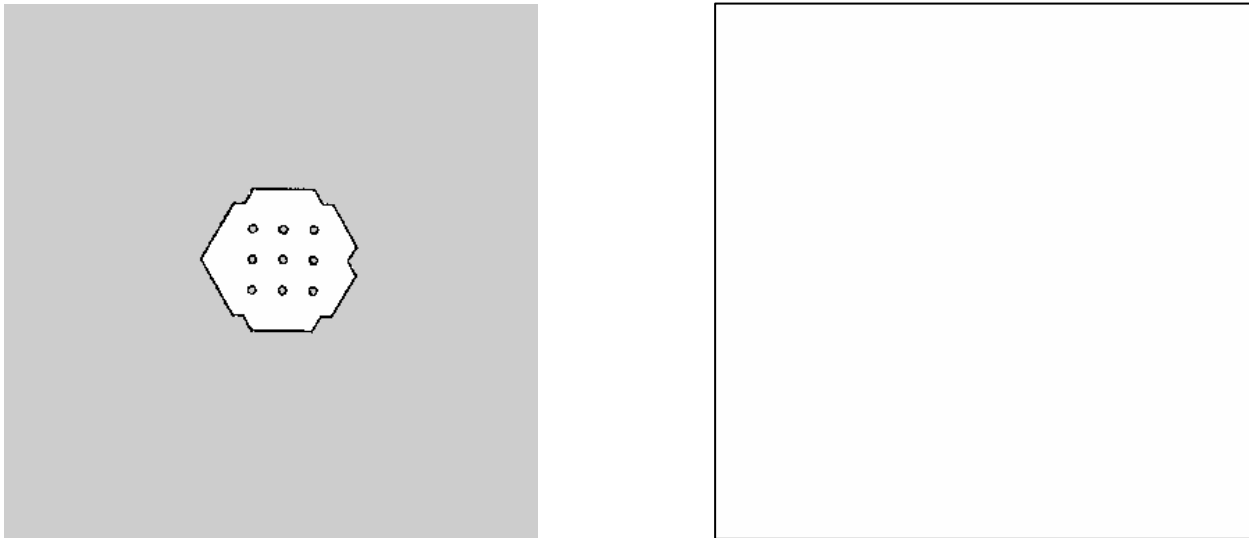
Another useful module is the Waypoint Follower. This module allows the robot to follow a list of predefined waypoints. It sends these waypoints as sequential navigation goals to the BT Navigator. At each waypoint, the robot can be programmed to perform additional actions, such as taking a picture, playing a sound, or waiting for a specific duration. The waypoint follower can be configured to either stop on failure or skip to the next waypoint, providing

flexibility in handling navigation errors. This is particularly useful in structured environments such as factories, warehouses, or guided tours.

Map Server

The Map Server is responsible for providing the map of the robot's environment. The map file itself is passed in via the `navigation2.launch.py` file. In our case, the parameters for the Map Server are empty, meaning no special configuration is being applied.

On the left, you can see our example of a map with obstacles, and on the right, a map without obstacles — basically a blank map.



These maps are based on grayscale images, where each pixel value has a specific meaning:

- 0 represents black, which means the space is occupied — there's an obstacle.
- 255 is white, meaning the area is free and navigable.
- Values between 1 and 254 are various shades of grey, indicating unknown or uncertain areas — usually due to limited sensor data.

This grayscale encoding is fundamental for navigation in Nav2. The robot uses this map to detect obstacles and safely plan paths through the environment.